

DynaFLUX: Implicit Dynamics-Preserving Reinforcement Learning for Topology-Free Influence Maximization

Daiyunke Zhang
Beihang University
Beijing, China
zhangdaiyunke@buaa.edu.cn

Ting Deng
Beihang University
Beijing, China
dengting@buaa.edu.cn

Tianchen Zhu*
Beihang University
Beijing, China
catezi@buaa.edu.cn

Shuai Ma
Beihang University
Beijing, China
mashuai@buaa.edu.cn

Daqing Li
Beihang University
Beijing, China
Civil Aviation University of China
Tianjin, China
daqingl@buaa.edu.cn

Mingtian Peng
TravelSky Technology Limited
Beijing, China
mtpeng@travelsky.com.cn

Feng Tian
TravelSky Technology Limited
Beijing, China
tianfeng@travelsky.com.cn

Abstract

Influence maximization (IM) aims to select a small set of seed nodes whose activation triggers a maximal cascade. Existing methods typically assume access to the network topology or a reliable surrogate, which is often unavailable in practice due to noisy, privacy-protected, or partially observed links. These settings pose two challenges: (1) the lack of explicit topology removes a key inductive constraint for diffusion modeling, undermining influence estimation, and (2) nonlinear, temporally dependent node interactions yield complex multivariate time series that hinder topology inference. We propose DynaFLUX, an end-to-end generative framework for IM under hidden topology. DynaFLUX learns a compact surrogate of latent dynamics directly from observed time series, and jointly optimizes a seed-selection policy via reinforcement learning. A self-attention pointer network captures long-range dependencies for seed generation, while an influence-prediction module infers a surrogate topology and uses Monte Carlo diffusion to provide policy-gradient rewards. Experiments show that DynaFLUX accurately identifies influential spreaders and consistently outperforms state-of-the-art baselines in unseen topology scenarios.

CCS Concepts

• Information systems → Social networks; • Computing methodologies → Reinforcement learning.

*Corresponding author.



This work is licensed under a Creative Commons Attribution 4.0 International License.
WWW '26, Dubai, United Arab Emirates
© 2026 Copyright held by the owner/author(s).
ACM ISBN 979-8-4007-2307-0/2026/04
<https://doi.org/10.1145/3774904.3792742>

Keywords

Influence Maximization; Reinforcement Learning; Topology-Free Graph Learning

ACM Reference Format:

Daiyunke Zhang, Ting Deng, Tianchen Zhu, Shuai Ma, Daqing Li, Mingtian Peng, and Feng Tian. 2026. DynaFLUX: Implicit Dynamics-Preserving Reinforcement Learning for Topology-Free Influence Maximization. In *Proceedings of the ACM Web Conference 2026 (WWW '26)*, April 13–17, 2026, Dubai, United Arab Emirates. ACM, New York, NY, USA, 12 pages. <https://doi.org/10.1145/3774904.3792742>

Resource Availability:

The source code of this paper has been made publicly available at <https://github.com/catezi/DynaFLUX>.

1 Introduction

Influence maximization is a central and lasting problem in social network research, which aims to select a small set of seed nodes whose activation triggers a cascade and maximizes spread [13]. Classic solutions typically presuppose access to the network topology and a known diffusion model, using this information to identify high-value seeds via greedy search, reverse sampling, or learning-based ranking schemes. However, many real-world social network applications do not admit reliable topological priors: the true interaction structure is often unobserved, privacy-restricted, or only indirectly observed through node-level time series. In such settings, we must infer latent topology from dynamics observations and then perform seed selection on that inferred representation.

Much of the prior work on influence maximization therefore assumes either complete knowledge of the underlying graph or a trustworthy surrogate graph. Traditional non-learning methods including greedy algorithms with Monte Carlo simulations [17], CELF optimization [17], and reverse influence sampling [8, 20, 32, 33] have been developed to trade off accuracy and scalability under explicit diffusion models. Recent learning-based methods use graph neural

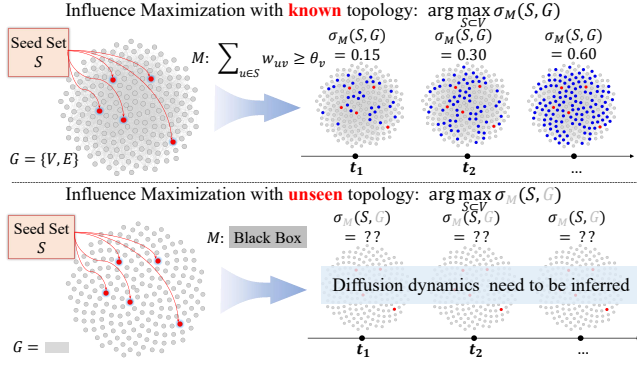


Figure 1: Illustration of the practical distinction between IM with a known topology and the unseen setting we target.

networks [15, 43], deep reinforcement learning [3], or learning-to-rank frameworks [14, 27] to select seeds when a graph structure is given or reliably estimated. Yet when the topology is unseen or proxy networks are noisy (missing or spurious links), topology-dependent methods degrade: estimated structures can diverge substantially from the true, latent connectivity and thereby misidentify influential spreaders [29]. As illustrated in Fig. 1, the contrast between known and unseen topologies highlights the practical challenges of influence maximization in unseen or noisy networks.

Addressing influence maximization in unseen or noisy networks raises two core technical challenges [4]. First, the absence of explicit topology removes an important inductive constraint for diffusion dynamics modeling, which can yield large errors in influence estimation if surrogate dynamics are poorly specified. Second, nonlinear interactions and temporal dependencies between nodes produce complex node-state time series that exceed the representational capacity of conventional sequence models and thereby impede accurate inference of latent topological structure [41].

To overcome these challenges, we propose DynaFLUX, an end-to-end generative framework for influence maximization that operates without knowing topologies. DynaFLUX learns a compact surrogate of the latent dynamics directly from limited multivariate time series and jointly optimizes a seed-selection policy—thereby enabling accurate influence prediction even when the true connectivity is unseen. Our contribution can be summarized as follows:

- We introduce a principled, end-to-end framework that infers both surrogate topology and implicit dynamics from time series data without assuming access to the ground-truth graph.
- We design a reinforcement learning-based policy that generates seed sets via a self-attention pointer network, which captures long-range dependencies in the learned dynamics.
- We develop an influence-prediction module that learns a surrogate topology from observed dynamics and estimates seed influence via Monte Carlo diffusion to provide policy-gradient rewards.
- We evaluate DynaFLUX on several real-world and synthetic benchmarks, demonstrating that it accurately identifies influential spreaders on challenging unseen topology settings, and outperforms baselines that rely on full topology or accessible dynamics.

2 Related work

2.1 Influence Maximization Methods

Influence maximization (IM) research can be categorized into classical combinatorial methods and recent learning-based approaches.

Combinatorial methods typically assume full access to the network topology and diffusion dynamics, and focus on designing scalable algorithms with provable approximation bounds. These include greedy algorithms [13, 17], sampling-based methods [2, 26, 34], and integer programming formulations, [11, 20].

Learning-based methods alleviate reliance on explicit topology or diffusion rules by training directly on graph data, including GNN-based models such as GLIE [28] and DREIM [22], multi-task frameworks such as IMINFECTOR [27], deep reinforcement learning approaches such as PIANO [19] and ToupleGDD [3], differentiable ranking-based formulations such as DeepIM [21], and higher-order message-passing architectures such as DeepSN [9]. Although these methods can reduce reliance on Monte Carlo simulations and can model complex dynamics, they often retain inductive biases through partial topology exposure or predefined embedding structures.

2.2 Time-Series-Based Topology Inference

Another line of work focuses on inferring latent connectivity or causal dependencies from temporal cascades, especially when only node-level activation times (e.g., infection times) are observed. These include maximum-likelihood methods under continuous-time diffusion models [7], multivariate Hawkes process-based approaches for capturing causal dependencies in high-frequency event data, such as minimum-message-length model selection for recovering Granger-causal graphs [10], and deep Hawkes models with attention mechanisms that relax linearity assumptions and enable instance-level causal inference [40].

2.3 Influence Maximization under Partially Observed Topologies

A parallel line of research investigates IM under incomplete network observability, where only partial topology is accessible via limited probing. Representative approaches include iterative probe-update-diffuse cycles like [24], learning-based estimators [42], subgraph construction via fixed-length random walks with local heuristic seed selection [38], Markov decision process formulation [12], influence approximation using ensembles of hypothetical graph completions [6], metadata-aware edge inference using GNNs and Siamese network [35], and curriculum-trained reinforcement agents [18]. In contrast, DynaFLUX neither reconstructs the graph nor estimates individual influence scores. Instead, it learns node embeddings directly from global temporal cascades, implicitly capturing connectivity patterns. This enables end-to-end seed selection solely from activation data, without access to network or diffusion parameters.

3 Preliminaries

Notations. Consider a complex system with an unobservable network topology $G = (V, E)$, represented by an $N \times N$ weighted adjacency matrix A with entries a_{ij} . A directed edge from node i to node j exists precisely when $a_{ij} > 0$; otherwise $a_{ij} = 0$. We further

assume that $\mathbf{A}$ is nonnegative ($a_{ij} \geq 0$) and time invariant, so that the underlying graph remains fixed.

Let $x_i(t)$ denote the state of node i at time t , and $\mathbf{x}(t) = (x_1(t), \dots, x_N(t))^T$ the vector of all the states of the nodes. Although the topology is hidden, we observe the time series data $\mathcal{D} = \{\mathbf{x}(t)\}_{t=0}^T$, which captures the network dynamics over the T steps.

Problem Formulation. Let $G = (V, E)$ be a directed graph and M be one of the following common diffusion models. (1) Independent Cascade (IC) model, where each active node u has one chance to activate neighbor v with probability p_{uv} . (2) Linear Threshold (LT) model, where each node v draws $\theta_v \sim U[0, 1]$ and becomes active when $\sum_{u \in S} w_{uv} \geq \theta_v$, subject to $\sum_{u: (u,v) \in E} w_{uv} \leq 1$. (3) Susceptible–Infected–Recovered (SIR) model, where each infected node u has one opportunity to infect each susceptible neighbor v with probability β , and upon making these transmission attempts u immediately transitions to the recovered state, in which it can neither be infected again nor further propagate the infection.

For a seed set $S \subseteq V$, denote by $\sigma_M(S, G)$ the expected number of activated nodes under M , and k the seed budget. The classical influence maximization (IM) problem is

$$S^* = \arg \max_{S \subseteq V, |S|=k} \sigma_M(S, G), \quad (1)$$

We then consider the influence maximization problem on an unseen topology. Here, G is hidden and only node-level activation timestamps $\mathcal{T}(T_{\text{obs}}) = \{t_v : v \in V, t_v \leq T_{\text{obs}}\}$ are observed, where T_{obs} denotes the observation window. A seeding policy π maps $\mathcal{T}$ to a set $S = \pi(\mathcal{T}) \subseteq V$ of size k . The objective becomes

$$\pi^* = \arg \max_{|\pi(\mathcal{T})|=k} \sigma_M(\pi(\mathcal{T}), G), \quad (2)$$

Our method takes, for each node v , an observation sequence $\{x_v(t) : t \in \mathcal{T}(T_{\text{obs}})\}$ over the window T_{obs} and then outputs a seed set S .

Reinforcement learning for Influence Maximization. In Influence Maximization (IM), obtaining sufficient labeled data to train parameters in a supervised manner is typically infeasible. Instead, we formulate IM as a reinforcement learning (RL) problem, where the goal is to learn an optimal policy, *i.e.*, a sequence of actions selecting seed nodes. RL studies how an agent interacts with its environment to choose actions based on the current state to maximize cumulative reward. Here, the agent learns to select a seed set under unknown network dynamics.

We model the decision process as a Markov Decision Process, represented by a 5-tuple $(\mathcal{S}, \mathcal{A}, P, R, \gamma)$: (1) *State* $\mathcal{S}_t$: Encoded as a $|V|$ -dimensional binary vector indicating which nodes have been selected (1 if $u \in \mathcal{S}_t$, else 0); a terminal state $\mathcal{S}_b$ occurs once b nodes have been chosen. (2) *Action* $\mathcal{A}$: Selecting a node $u \in \bar{\mathcal{S}}_t$ (the complement of $\mathcal{S}_t$) as the next seed, represented by the its embedding. (3) *Transition* P : Choosing u flips its activation bit in $\mathcal{S}_t$ from 0 to 1, producing the next state $\mathcal{S}_{t+1}$. (4) *Reward* R : The immediate reward for selecting u at state $\mathcal{S}_t$ is $r(\mathcal{S}_t, u) = \sigma(\mathcal{S}_t \cup \{u\}) - \sigma(\mathcal{S}_t)$ and $r(\emptyset) = 0$. Consequently, the cumulative reward at the terminal state $\mathcal{S}_T$ equals the total expected spread $R = \sum_{t=0}^T r(\mathcal{S}_t, u_t) = \sigma(\mathcal{S}_T)$. Thus, maximizing the agent’s cumulative reward directly corresponds to maximizing the influence spread of the selected seed set. (5) *Discount factor* $\gamma \in [0, 1]$: A parameter that determines how future rewards are weighted relative to immediate rewards.

4 Methodology

In this section, we present the DynaFLUX framework, which predicts network-dynamics evolution and identifies an initial seed set that maximizes influence spread without topology priors.

We formulate seed selection as a Markov decision process optimized via policy gradients, comprising three modules: (1) **Generative Seed Selection Policy**: Iteratively proposes seed sets from limited time-series observations of network dynamics. (2) **Monte Carlo Influence Reward Prediction**: Infers a latent proxy topology and employs Monte Carlo diffusion simulations to estimate each seed set’s spread. (3) **Policy Gradient Optimization**: Refines seed proposals in a continuous action space to maximize influence propagation. Fig. 2 illustrates the DynaFLUX architecture.

4.1 Generative Seed-Selection Policy

Since the topology is unknown, the policy module ingests only node-wise sequential observations. For each node v_i , let $x_i \in \mathbb{R}^T$ denote its observation vector, which is projected into $\mathbb{R}^d$ by a shared linear mapping: $h_i^{(0)} = W_e x_i$. The module then encodes these embeddings through L self-attention layers [36] to infer implicit network dynamics and produce low-dimensional representations. We employ self-attention to model long-range temporal dependencies arising from cascading diffusion and critical transitions in complex networks—phenomena that classical sequence models often miss.

At layer ℓ , form $H^{(\ell)} = [h_1^{(\ell)}, \dots, h_N^{(\ell)}]^\top$ and compute

$$Q^{(\ell)} = H^{(\ell)} W_q^{(\ell)}, \quad K^{(\ell)} = H^{(\ell)} W_k^{(\ell)}, \quad V^{(\ell)} = H^{(\ell)} W_v^{(\ell)}, \quad (3)$$

where $W_q^{(\ell)}, W_k^{(\ell)}, W_v^{(\ell)} \in \mathbb{R}^{d \times d}$. The attention matrix is

$$A^{(\ell)} = \text{softmax}(Q^{(\ell)} (K^{(\ell)})^\top / \sqrt{d}). \quad (4)$$

Embeddings are updated via residual connections, layer normalization, and a feed-forward network:

$$\tilde{H}^{(\ell)} = \text{LayerNorm}(H^{(\ell)} + A^{(\ell)} V^{(\ell)}), \quad (5)$$

$$H^{(\ell+1)} = \text{LayerNorm}(\tilde{H}^{(\ell)} + \text{FFN}(\tilde{H}^{(\ell)})). \quad (6)$$

After L layers, the final embeddings $H^L = \{h_i^L\}$ capture both temporal correlations and implicit topology. A decoder then maps these embeddings into a node selection distribution, from which the highest-probability node is selected.

PROPOSITION 4.1. *Each layer’s attention weights*

$$A_{ij}^{(\ell)} = \frac{\exp(\langle Q_i^{(\ell)}, K_j^{(\ell)} \rangle / \sqrt{d})}{\sum_{k=1}^N \exp(\langle Q_i^{(\ell)}, K_k^{(\ell)} \rangle / \sqrt{d})} \quad (7)$$

form an $N \times N$ “soft” adjacency matrix, enabling implicit edge inference without explicit graph reconstruction.

To generate the seed set, we perform a k -step sequential decision process. Inspired by pointer networks, our decoder is designed to flexibly adapt to networks of various sizes. At step t , we update the context using the embedding s_{t-1} of the previously selected node v_{t-1} . Given node embeddings $\{h_{t-1}^{(i)}\}$ and candidate mask $\mathcal{M}_t \in \mathbb{R}^{d \times 1}$, we compute:

$$s_t^i = f(h_{t-1}^{(i)}, s_{t-1}), \quad u_t^i = W_d s_t^i, \quad (8)$$

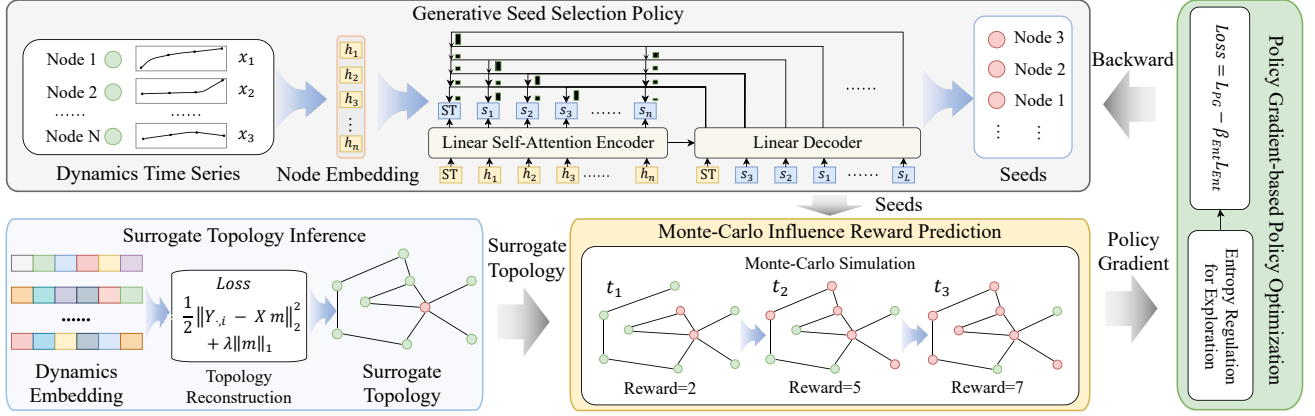


Figure 2: Data flow in DynaFLUX: (1) input activation time-series x_i for each node; (2) encoder transforms $\{x_i\}$ into embeddings $\{h_i\}$ via temporal-correlation attention; (3) sequential policy uses $\{h_i\}$ to sample a seed sequence $(v_1, \dots, v_k)$; (4) Monte Carlo diffusion simulations compute marginal rewards r_t ; (5) policy gradients derived from $\{r_t\}$ update encoder and policy parameters.

with $W_d \in \mathbb{R}^{1 \times d}$. The selection distribution over unchosen nodes is

$$\pi^{(t)}(i) = \frac{\exp(u_i^t)}{\exp(u_i) \mathcal{M}_t}. \quad (9)$$

During training, $\pi^{(t)}$ samples nodes from an expanding candidate set (initially heuristic, then all nodes); at inference, it selects the node with the maximum probability.

4.2 Monte Carlo Influence Reward Prediction

To evaluate our seed selection policy and establish an optimization objective, we ordinarily perform Monte Carlo diffusion simulations to measure influence spread. However, when the true graph $\mathcal{G}$ is unavailable, this approach becomes infeasible.

Instead, we infer a surrogate adjacency matrix $\hat{M} \in \mathbb{R}^{N \times N}$ from the observed state sequences $\{x^{(0)}, \dots, x^{(T)}\}$ using sparse linear regression framework with Lasso regularization.

In each Monte Carlo trial on G_{sim} (or on $\mathcal{G}$ if available), we initiate diffusion from the current seed set $S_{t-1} = \{v_1, \dots, v_{t-1}\}$ and evaluate coverage $\sigma(S)$. The marginal gain of adding v_t is then

$$r_t = \frac{1}{R} \sum_{r=1}^R \left[\sigma(\{v_1, \dots, v_t\}) - \sigma(\{v_1, \dots, v_{t-1}\}) \right], \quad (10)$$

averaged over R independent runs.

For more sufficient comparison, we evaluate two variants in our experiments: (1) DynaFLUX: Monte Carlo reward estimation using the Lasso-inferred proxy topology G_{sim} . (2) DynaFLUX+: Monte Carlo reward estimation using the true topology.

4.3 Policy Optimization over Policy Gradient

Because Monte Carlo-estimated marginal gains are non-differentiable with respect to the parameters of the policy, we adopt policy gradient to optimize the encoder, decoder, and related neural parameters, as detailed in Algorithm 1. Let b_t be a baseline and define the advantage $A_t = r_t - b_t$. The expected return is $J(\theta) = \mathbb{E}_{S \sim \pi_\theta} \left[\sum_{t=1}^k r_t \right]$ [39]

and the policy gradient is given by [31]:

$$\nabla_\theta J(\theta) = \mathbb{E}_{S \sim \pi_\theta} \left[\sum_{t=1}^k r_t \nabla_\theta \log \pi^{(t)}(v_t) \right]. \quad (11)$$

To promote exploration in early training and avoid premature convergence, we introduce an entropy regularizer [25]. The resulting entropy-regularized REINFORCE loss is

$$L_{\text{PG}} = - \sum_{t=1}^k A_t \log \pi^{(t)}(v_t), \quad L = L_{\text{PG}} - \beta_{\text{ent}} \sum_{t=1}^k H(\pi^{(t)}), \quad (12)$$

where $H(\pi) = - \sum_i \pi(i) \log \pi(i)$, and the entropy coefficient β_{ent} decays over training to balance exploration and convergence.

4.4 Complexity Analysis

We analyze the per-iteration cost of DynaFLUX by decomposing its components. First, the encoder consists of L self-attention layers, each computing an $N \times N$ attention matrix by projecting and multiplying $Q, K \in \mathbb{R}^{N \times d}$, costing $O(N^2 d)$ for the dot-products and softmax, plus $O(Nd^2)$ for the feed-forward sublayers. Summed over L layers, this yields $O(L(N^2 d + Nd^2))$. Second, the sequential policy executes k masked softmax operations, each requiring $O(Nd)$ to score and normalize N embeddings, leading to $O(kNd)$. Third, reward estimation runs R Monte Carlo diffusion simulations, each $O(|E|)$, totaling $O(R|E|)$. Hence the per-iteration complexity is $O(LN^2 d + LNd^2 + kNd + R|E|)$.

As typically $N \gg d$ and L, d, R are constants with $|E| \leq N^2$, the overall complexity is dominated by $O(N^2)$, i.e., quadratic in the number of nodes. Furthermore, any structure-based IM method under hidden topology must consider $\binom{N}{2}$ candidate links, establishing an $\Omega(N^2)$ lower bound.

5 Experiments

In this section, we evaluate DynaFLUX under the Independent Cascade (IC), Linear Threshold (LT) and Susceptible-Infected-Susceptible (SIS) diffusion models. We compare it with state-of-the-art baselines

Algorithm 1 RL Training for DynaFLUX

Require: Graph G , time-series $x \in \mathbb{R}^{N \times T}$, policy π_θ , seed rate r , diffusion model, MC runs per step R , total epochs E , baseline decay α

Ensure: Trained policy parameters θ

```

1:  $N \leftarrow$  number of nodes implied by  $x$ 
2:  $k \leftarrow \lceil N \cdot r \rceil$ 
3: Initialize running baseline  $\text{baseline}[1:k] \leftarrow 0$ 
4: for epoch  $e = 1$  to  $E$  do
5:    $\text{selected}, \log\_probs, probs \leftarrow \pi_\theta(x, k)$ 
6:   Initialize cumulative influence  $\text{Inf} \leftarrow 0$ 
7:   Initialize reward list  $\text{reward}[1:k] \leftarrow []$ 
8:   for step  $t = 1$  to  $k$  do
9:      $S_t \leftarrow \text{selected}[1:t]$ 
10:     $\text{Inf}' \leftarrow \text{simulate}(S_t, R, \text{diffusion model})$ 
11:     $\text{reward}[t] \leftarrow \frac{\text{Inf}' - \text{Inf}}{N}$ 
12:    Update cumulative influence:  $\text{Inf} \leftarrow \text{Inf}'$ 
13:   end for
14:   Compute advantages:
15:   for  $t = 1$  to  $k$  do
16:     if  $e == 1$  then
17:        $\text{baseline}[t] \leftarrow \text{reward}[t]$ 
18:     else
19:        $\text{baseline}[t] \leftarrow \alpha \cdot \text{baseline}[t] + (1 - \alpha) \cdot \text{reward}[t]$ 
20:     end if
21:      $\text{Advantage}[t] \leftarrow (\text{reward}[t] - \text{baseline}[t]) \cdot \text{scale}$ 
22:   end for
23:    $L_{PG} \leftarrow -\sum_{i=1}^k \log\_probs[i] \cdot \text{Advantage}[i]$ 
24:    $H \leftarrow -\sum_i probs[i] \log(probs[i] + \epsilon)$ 
25:    $L \leftarrow L_{PG} - \beta_{ent} \cdot H$ 
26:   Update policy parameters by taking a gradient step on  $L$ 
27: end for
28: Save final policy  $\pi_\theta$ 
29: return  $\pi_\theta$ 

```

on five real-world networks and one synthetic benchmark, using the same diffusion setting as DeepIM. We further conduct sensitivity analyses, an attention ablation study, and a qualitative case study to validate the effectiveness and robustness of our approach.

5.1 Experimental Setup

Datasets. We evaluate DynaFLUX on six networks: Jazz, CoraML, Power Grid (PG), Network Science (NS), Digg, and synthetic Erdős–Rényi graphs (SD). Detailed dataset descriptions are provided in the appendix.

Baselines. We compare DynaFLUX with (1) *traditional* influence maximization models, including IMM [33], OPIM [32] and SubSIM [8]; (2) *learning-based* methods, including IMINFECTOR [27], PIANO [19], ToupleGDD [3], DeepIM [21] and DeepSN [9]; and (3) the variant DynaFLUX+, which uses Monte Carlo reward estimation based on the ground-truth topology. The experimental protocol and configuration strictly follow those of DeepIM [21] to ensure consistent and comparability. Note that under the SIS model, we exclude DeepSN from quantitative comparisons, because its experimental protocol in the released code is not aligned with

the DeepIM configuration we adopt; therefore, the results are not directly comparable. For reproducibility, all implementation and simulation details are deferred to the appendix.

Implementation. We trained DynaFLUX and DynaFLUX+ on synthetic graphs and evaluated the learned policy on the six networks. For both training and evaluation, node-level time series data are generated from the underlying graphs via a diffusion–pulsing process. The detailed construction procedure, including update rules and noise injection, is provided in the appendix.

5.2 Main Results

We evaluate the performance of DynaFLUX by comparing its average final spread with a diverse set of baselines across all datasets under varying seed set sizes (1%, 5%, 10%, and 20%). For each setting, we run 100 Monte Carlo simulations and report the average proportion of nodes activated at the end of the diffusion process.

As shown in Table 1, DynaFLUX consistently outperforms both heuristic methods (IMM, OPIM and SubSIM) and learning-based approaches (IMINFECTOR, PIANO, ToupleGDD, DeepIM and DeepSN) on nearly all networks under the IC and LT models, and achieves superior performance on some networks under the SIS model.

Specifically, under the IC and LT models, we found the following. (1) Compared to the strongest non-DynaFLUX baseline at each seed budget, DynaFLUX improves the expected node coverage by **+15.8**, **+13.6**, **+9.5**, and **+5.8** percentage points under IC, and by **+13.3**, **+13.9**, **+14.5**, and **+4.5** percentage points under LT, at seed budgets of 1%, 5%, 10%, and 20%, respectively. These are absolute coverage gains (in percentage points) averaged across six networks. (2) Although improvements are not uniformly positive across all datasets, the averages remain consistently positive, with the largest benefits observed in the small-to-moderate budget regime (1%–10%), highlighting DynaFLUX’s ability to identify highly influential seeds with minimal resources. (3) As the seed set size increases, performance gaps naturally narrow due to diffusion saturation; nevertheless, DynaFLUX continues to achieve meaningful additional coverage even at 20% budget level. (4) DynaFLUX+, which uses the true network for reward guidance, consistently outperforms the surrogate-based DynaFLUX on almost all networks, confirming that exact topology provides stronger training signals, even though surrogate guidance already delivers strong robustness and generalization.

The consistent gains observed under the IC and LT models stem from the core design of DynaFLUX. Instead of relying on the exact underlying topology, DynaFLUX learns its policy from a surrogate graph whose reward signals remain directionally aligned with those of the true network. This decoupling between reward generation and real topology reduces the need for explicit structural supervision and enhances the robustness to noise, missing edges, and dynamic fluctuations commonly observed in real diffusion data. As the surrogate graph is optimized to preserve reward consistency rather than strict topological fidelity, the learned policy generalizes well across diverse graph structures and diffusion regimes.

However, on networks such as Digg, we observe that using the true graph for reward computation (DynaFLUX+) can perform slightly worse than the surrogate guidance (DynaFLUX) at larger seed budgets (10%–20%). This is likely due to the high-variance and non-smooth reward landscapes of the true topology, whereas the

Table 1: Influence spread (%) under the IC, LT and SIS models. (Bold (resp. underline) are the best (resp. second-best) results.)

Method	Cora-ML (IC)				Network Science (IC)				Power Grid (IC)				Jazz (IC)				Synthetic (IC)				Digg (IC)			
	1%	5%	10%	20%	1%	5%	10%	20%	1%	5%	10%	20%	1%	5%	10%	20%	1%	5%	10%	20%	1%	5%	10%	20%
IMM	8.1	26.2	37.3	50.2	5.2	16.8	27.0	45.7	4.3	17.4	31.5	51.1	2.6	20.1	31.4	42.8	9.2	26.2	36.3	51.6	7.4	18.4	32.8	49.6
OPIM	13.4	26.9	37.4	50.9	6.6	19.4	28.9	48.6	5.7	17.7	29.7	50.1	2.4	20.1	34.4	46.8	9.6	25.3	36.6	51.7	7.6	18.5	32.9	48.9
SubSIM	10.1	25.7	36.8	51.1	4.8	15.4	27.9	44.8	4.6	19.2	31.7	50.2	3.6	18.8	37.6	44.7	9.5	26.7	36.5	51.5	7.5	18.9	33.3	49.4
IMINFECTOR	9.6	26.8	37.7	50.6	5.4	17.9	27.8	47.6	5.4	18.2	31.6	50.9	3.6	19.7	37.5	45.9	9.1	26.2	36.1	51.5	7.9	18.6	33.5	49.8
PIANO	9.8	25.2	37.4	51.1	4.7	16.3	27.1	47.2	5.3	18.1	31.7	50.2	2.2	19.2	36.6	43.2	9.1	26.4	36.2	51.6	—	—	—	—
ToupleGDD	10.6	27.5	38.5	51.5	6.3	17.8	28.3	50.5	5.4	19.3	31.6	51.3	3.3	20.4	37.2	45.7	9.5	26.8	37.1	51.4	—	—	—	—
DeepIMs	13.6	27.7	38.5	51.8	6.9	19.1	29.3	50.5	5.9	20.2	31.7	51.5	3.8	21.4	38.9	47.1	10.2	26.8	37.5	51.8	7.9	18.8	33.7	50.3
DeepIM	14.1	28.1	39.6	52.4	7.8	20.9	31.5	51.2	6.3	21.0	32.5	52.4	4.9	23.3	41.5	49.9	11.6	27.4	38.7	52.1	8.4	19.3	34.2	51.3
DeepSN	14.2	30.1	42.3	58.4	<u>7.9</u>	18.6	30.0	51.2	7.1	22.4	37.4	57.4	8.5	26.9	41.6	53.8	10.6	27.8	38.8	52.8	8.9	19.5	35.2	52.8
DynaFLUX	<u>35.3</u>	<u>46.8</u>	<u>55.5</u>	<u>65.4</u>	<u>7.8</u>	<u>21.1</u>	<u>29.6</u>	<u>41.7</u>	<u>9.0</u>	<u>27.4</u>	<u>41.3</u>	<u>58.6</u>	<u>26.0</u>	<u>37.3</u>	<u>41.7</u>	<u>56.4</u>	14.4	30.4	41.1	<u>54.4</u>	<u>60.3</u>	<u>66.2</u>	<u>74.8</u>	84.6
DynaFLUX+	38.4	47.4	56.5	68.2	9.3	23.4	32.9	45.1	9.1	29.0	43.1	61.6	26.3	39.7	45.2	60.8	<u>14.1</u>	<u>30.3</u>	<u>41.4</u>	<u>54.5</u>	60.8	68.6	74.1	<u>78.4</u>

Method	Cora-ML (LT)				Network Science (LT)				Power Grid (LT)				Jazz (LT)				Synthetic (LT)				Digg (LT)			
	1%	5%	10%	20%	1%	5%	10%	20%	1%	5%	10%	20%	1%	5%	10%	20%	1%	5%	10%	20%	1%	5%	10%	20%
IMM	1.7	34.8	52.2	66.4	2.5	11.9	18.1	33.6	4.6	19.9	31.7	56.9	1.4	5.7	13.4	24.5	1.4	5.2	13.1	66.9	2.4	10.8	37.4	55.6
OPIM	2.3	36.9	51.2	71.5	1.6	12.0	18.8	34.1	4.4	21.6	29.4	55.5	1.4	6.9	12.6	20.9	1.3	5.2	12.6	62.1	2.1	11.3	38.2	57.1
SubSIM	1.7	33.6	54.7	70.1	1.8	10.4	19.2	34.1	4.5	21.1	31.2	57.4	1.4	5.9	11.4	21.2	1.4	5.5	13.1	69.6	2.4	11.3	37.9	56.9
IMINFECTOR	2.1	33.9	51.3	70.6	2.1	11.8	18.7	34.5	4.2	21.3	31.6	56.2	1.4	6.2	13.5	22.8	1.3	5.2	12.9	67.4	2.2	11.1	38.9	58.7
PIANO	2.1	33.5	53.3	69.8	2.1	11.3	19.1	33.9	4.3	21.3	31.4	57.1	1.1	6.2	12.1	22.4	1.2	5.2	12.9	67.4	—	—	—	—
ToupleGDD	2.3	36.2	54.5	70.9	2.8	12.4	19.8	34.6	4.8	21.9	32.6	58.1	1.4	6.5	12.9	23.6	1.3	5.5	13.4	70.2	—	—	—	—
DeepIMs	10.7	65.6	75.1	85.2	3.5	14.6	23.8	37.8	5.1	22.9	40.3	65.1	1.4	6.5	14.2	85.3	<u>1.5</u>	6.0	14.2	90.3	3.1	13.3	39.2	67.9
DeepIM	13.4	69.2	83.5	94.1	4.1	16.6	26.7	41.5	6.3	24.4	46.8	71.7	<u>1.9</u>	6.5	<u>16.4</u>	99.1	<u>1.5</u>	6.5	15.5	99.9	3.5	15.9	41.3	76.2
DeepSN	7.9	46.4	72.8	93.8	3.6	15.5	27.8	51.8	5.2	23.8	40.3	68.1	2.0	6.7	14.9	96.9	1.6	5.8	13.5	99.9	3.2	16.1	41.7	72.1
DynaFLUX	<u>15.1</u>	73.8	<u>89.5</u>	<u>94.4</u>	<u>5.0</u>	<u>20.3</u>	<u>30.8</u>	45.0	<u>8.3</u>	<u>31.5</u>	<u>58.3</u>	<u>83.2</u>	1.5	<u>7.1</u>	36.9	99.9	1.0	5.2	11.2	99.9	<u>79.6</u>	<u>85.4</u>	91.9	97.1
DynaFLUX+	15.3	73.7	90.0	96.2	6.0	23.7	34.8	49.5	8.7	34.9	62.7	88.4	1.6	7.6	14.2	99.9	1.1	5.3	11.4	99.9	80.9	87.6	90.3	<u>91.7</u>

Method	Cora-ML (SIS)				Network Science (SIS)				Power Grid (SIS)				Jazz (SIS)				Synthetic (SIS)				Digg (SIS)			
	1%	5%	10%	20%	1%	5%	10%	20%	1%	5%	10%	20%	1%	5%	10%	20%	1%	5%	10%	20%	1%	5%	10%	20%
IMM	2.0	9.5	15.4	27.6	1.3	5.6	12.2	22.1	1.1	5.6	11.0	22.9	7.6	37.8	55.6	67.1	2.7	12.6	20.9	37.7	2.5	9.4	16.3	32.6
OPIM	2.3	9.3	16.2	27.2	1.4	5.9	13.0	22.1	1.2	5.9	11.1	22.4	8.2	35.1	56.8	68.3	2.8	12.5	20.2	36.1	2.3	9.3	16.5	32.3
SubSIM	2.3	9.2	16.9	28.8	1.5	5.6	12.2	23.3	1.2	5.6	11.4	21.9	2.9	30.1	53.8	67.0	3.2	14.4	24.5	39.1	2.5	9.5	16.1	32.3
IMINFECTOR	2.1	9.4	16.1	27.9	1.7	5.8	12.4	22.3	1.3	5.5	10.2	23.1	8.8	35.4	54.8	66.2	2.5	12.4	20.5	36.6	2.3	9.1	16.4	32.4
DeepIM	<u>7.1</u>	16.1	<u>21.9</u>	<u>30.8</u>	<u>2.7</u>	<u>8.7</u>	15.1	25.1	<u>1.9</u>	<u>7.6</u>	<u>13.3</u>	<u>23.8</u>	27.1	<u>57.1</u>	68.1	74.1	3.2	14.4	24.5	39.1	5.6	11.4	18.8	36.3
DynaFLUX	6.9	15.2	20.8	29.9	2.3	7.0	12.4	22.7	1.8	6.8	12.2	22.3	30.6	56.4	64.5	70.3	3.0	12.6	22.2	37.0	<u>16.5</u>	18.0	21.1	28.5
DynaFLUX+	7.3	<u>15.9</u>	22.5	31.9	2.8	8.8	<u>14.1</u>	<u>24.2</u>	2.3	7.6	13.7	25.4	35.8	58.9	<u>64.7</u>	68.6	3.2	<u>13.5</u>	<u>23.6</u>	<u>38.4</u>	17.1	20.4	23.7	<u>31.7</u>

learned surrogate implicitly regularizes the training signal, yielding smoother gradients and more stable policy optimization.

Under the SIS model, where contagion is recurrent and temporal dynamics are nonlinear, the advantage of DynaFLUX becomes more modest, yielding average coverage gains of **+1.0**, **+0.9**, **+0.5**, and **−0.3** percentage points at budgets of 1%, 5%, 10%, and 20%, respectively. This smaller margin reflects the greater difficulty of inferring higher-order and nonlinear structural dependencies solely from time-series activations. Nonetheless, DynaFLUX still matches or surpasses strong baselines on several networks, demonstrating its generalization capability. Improving performance under nonlinear propagation regimes remains an key direction for future research.

5.3 Sensitivity Analysis

We next evaluate the impacts of temporal sequence length and training network type on the performance of DynaFLUX.

Temporal sequence length. Figure 3 shows the impact of varying input sequence lengths T (i.e., the number of timestamps per node) on the influence achieved by DynaFLUX+ across four real-world networks under the IC model. When T is small, DynaFLUX+ cannot adequately capture propagation dynamics, which limits the influence scores to relatively low levels. As T increases to around 20, the influence improves rapidly, after which the scores plateau. Overall,

these results indicate that influence scores remain relatively stable for moderate to large values of T , suggesting that a moderate historical window is sufficient for robust and efficient inference.

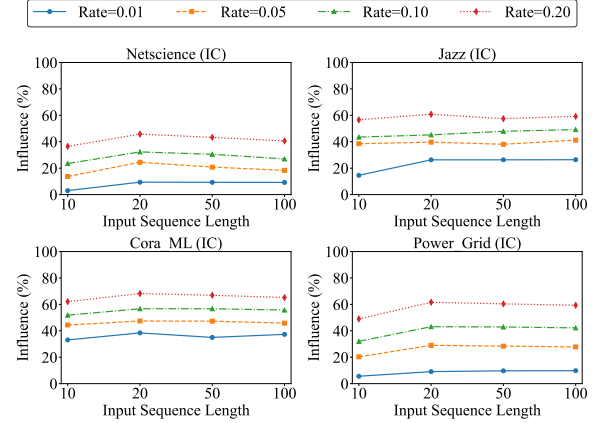


Figure 3: Sensitivity to temporal sequence length.

Training network type. Unlike DeepIM, which samples $\{(S, y)\}$ pairs from real networks, DynaFLUX+ is trained on simulated influence cascades over synthetic graphs. Specifically, we simulate cascades on Barabási–Albert (BA) [1], Erdős–Rényi (ER) [5],

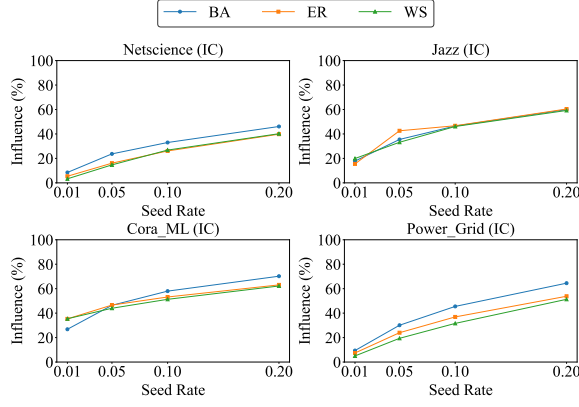


Figure 4: Sensitivity to training network type (BA, ER, WS).

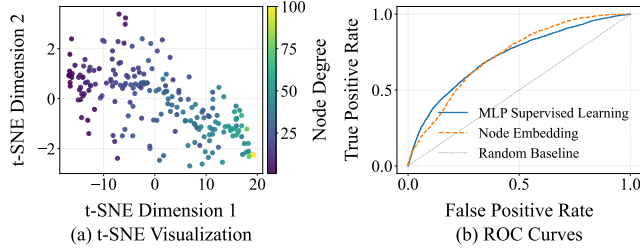


Figure 5: (a) t-SNE of DynaFLUX embeddings on Jazz, colored by node degree; (b) ROC curves for link prediction using a supervised MLP on raw time series versus cosine similarity of embeddings, with corresponding AUC values.

and Watts–Strogatz (WS) [37] graphs, and train a separate DynaFLUX+ instances on each. We then evaluate all three models under the IC model on four real-world networks (Network Science, Jazz, Cora-ML, Power Grid). The final influence spread differs by only a few percentage points across all seed ratios, with the BA-trained model showing a slight overall advantage (see Figure 4).

These results demonstrate that DynaFLUX+ is robust and largely insensitive to the choice of synthetic training topology. Therefore, except for the sensitivity analysis on the training network type, all models are trained on BA synthetic graphs.

5.4 Ablation Study

We perform an ablation study to isolate the effect of our temporal-correlation attention module. On five networks (Network Science, Jazz, Cora_ML, Power_Grid, Synthetic) and at seed ratios of 1%, 5%, 10%, and 20%, we compare three variants under the IC and LT diffusion models: MLP (DynaFLUX+ without attention), an LSTM-based baseline, and DynaFLUX+ (with attention).

Table 2 reports the influence spread percentages. In almost every setting, DynaFLUX+ yields the highest spread, with the largest gains observed on Jazz and Synthetic. This is attributed to the attention mechanism’s ability to dynamically weight pairwise temporal correlations, embeddings that emphasize coherent diffusion paths while suppressing noisy or weakly correlated activations. Without attention, node representations remain static projections of

their time series and fail to capture inter-node dynamics, leading to suboptimal seed selection. These results confirm that temporal-correlation attention effectively identifies and prioritizes nodes whose activation timing most strongly drives overall diffusion.

5.5 Validation of Reward Guidance

In DynaFLUX, both the true and Lasso-inferred synthetic graphs are used only during training to compute the Monte Carlo reward signal. Although these two topologies may differ substantially, what ultimately matters is that the estimated reward direction guides the policy correctly. To verify this, we sampled $n=100$ seed sets $\{S_i\}_{i=1}^n$ during RL training and measured their influence spreads on the true graph (i.e., $r_i = \sigma_{\text{true}}(S_i)$) and on the Lasso-inferred graph (i.e., $\hat{r}_i = \sigma_{\text{sim}}(S_i)$). Agreement was quantified using Pearson’s r , Spearman’s ρ , Kendall’s τ , and the Pairwise Ranking Consistency (PRC):

$$\text{PRC} = \frac{|\{(i, j) : \text{sgn}(\hat{r}_i - \hat{r}_j) = \text{sgn}(r_i - r_j)\}|}{\binom{n}{2}}, \quad (13)$$

where $\binom{n}{2}$ is the number of unordered pair from the n samples.

To show the necessity of the Lasso-based reconstruction, we compared it with three alternative correlation-based methods: (a) an ϵ -thresholded graph (Epsilon); (b) a per-node k -nearest-neighbors graph (KNN); and (c) a probabilistic graph (Prob) that samples edges with probability proportional to the correlation magnitude.

Each reconstructed graph was used to compute the simulated reward $\hat{r}_i$, and its alignment with the true reward r_i was assessed using the same metrics. The Lasso-based reconstruction shows the highest consistency in both magnitude and ranking. Table 3 summarizes the results, showing that despite topological discrepancies, the reward estimates remain well-aligned in both magnitude and ranking, ensuring reliable directional guidance for policy learning.

Detailed parameter settings and implementation for Epsilon, KNN, and Prob are provided in the appendix.

5.6 Implicit Structure Encoding Evaluation

To show that DynaFLUX’s node embeddings capture the underlying network structure, we conduct a link-prediction experiment on Jazz.

We visualize the learned embeddings $\{h_i\}_{i=1}^N$ using t-SNE, coloring each point by its degree d_i . Figure 5(a) shows that high-degree nodes cluster together, while low-degree nodes form distinct groups, indicating that embeddings implicitly encode structural centrality.

We next compare two link prediction approaches. The first is a supervised MLP that takes as input the concatenation $[x_i, x_j]$ of the raw time-series for nodes i and j , trained on synthetic graphs and evaluated on Jazz. The second uses the cosine similarity of DynaFLUX’s node embeddings, $S_{ij} = \frac{h_i^T h_j}{\|h_i\| \|h_j\|}$, as the link score. For all pairs $1 \leq i < j \leq N$, the ground-truth labels are defined as

$$y_{ij} = \begin{cases} 1, & (i, j) \in E, \\ 0, & \text{otherwise,} \end{cases} \quad (14)$$

and set $s_{ij} = S_{ij}$. We then compute the ROC curve (FPR, TPR) and the area under the curve $\text{AUC} = \text{auc}(\text{FPR}, \text{TPR})$ for both methods (Figure 5(b)). The MLP achieves $\text{AUC} = 0.722$, while the embedding-based predictor reaches $\text{AUC} = 0.726$, indicating that DynaFLUX’s embeddings capture degree-based patterns and provide effective structural features for downstream tasks such as link prediction.

Table 2: Ablation study: influence spread (%) of MLP, LSTM, and DynaFLUX+ under IC and LT diffusion (Best shown in bold).

Method	Cora_ML (IC)				Network Science (IC)				Power Grid (IC)				Jazz (IC)				Synthetic (IC)			
	1%	5%	10%	20%	1%	5%	10%	20%	1%	5%	10%	20%	1%	5%	10%	20%	1%	5%	10%	20%
MLP	34.7	44.4	52.3	67.3	7.3	20.3	30.4	42.4	8.4	27.7	41.2	59.9	25.7	38.5	44.2	55.9	13.0	30.2	41.0	54.4
LSTM	36.2	47.5	56.3	66.5	8.8	19.6	28.9	41.9	9.4	27.5	41.1	59.5	26.3	41.0	46.4	56.9	13.9	30.7	41.3	54.3
DynaFLUX+	38.4	47.4	56.7	68.2	9.3	23.4	32.9	45.1	9.1	29.0	43.1	61.6	26.3	39.7	45.2	60.8	14.1	30.3	41.4	54.5

Method	Cora_ML (LT)				Network Science (LT)				Power Grid (LT)				Jazz (LT)				Synthetic (LT)			
	1%	5%	10%	20%	1%	5%	10%	20%	1%	5%	10%	20%	1%	5%	10%	20%	1%	5%	10%	20%
MLP	15.3	72.8	87.1	95.8	3.8	21.3	32.6	49.3	6.9	29.3	55.6	85.3	1.5	7.0	14.0	53.5	1.0	5.2	10.8	30.3
LSTM	15.2	77.0	88.0	94.5	5.8	19.0	30.0	45.3	8.3	31.7	60.5	83.2	1.5	7.1	14.1	99.9	1.0	5.2	11.3	99.9
DynaFLUX+	15.3	73.7	90.0	96.2	6.0	23.7	34.8	49.5	8.7	34.9	62.7	88.4	1.6	7.6	14.2	99.9	1.0	5.3	11.4	99.9

Table 3: Agreement between predicted ($\hat{r}$) and true (r) influence spreads under four graph reconstructions.

Metric	Lasso	Epsilon	KNN	Prob
Pearson's r	0.606	0.145	0.376	0.214
p -value	2.42×10^{-11}	0.1495	1.18×10^{-4}	0.0327
Spearman's ρ	0.675	0.175	0.350	0.247
p -value	1.41×10^{-14}	0.0821	3.59×10^{-4}	0.0132
Kendall's τ	0.493	0.114	0.248	0.158
p -value	3.76×10^{-13}	0.0924	2.61×10^{-4}	0.0203
PRC (%)	74.5	55.5	62.2	57.7

Table 4: Structural metrics of 20.2%-seed sets on Jazz: average core number, degree centrality, betweenness centrality, neighbor Jaccard overlap, and boundary-node ratio.

Model	$\bar{c}$	$\overline{DC}$	$\overline{BC}$	$\bar{J}$	BR (%)
DynaFLUX	22.27	0.2591	0.0202	0.2210	92.5
CELf	19.30	0.1709	0.00745	0.1263	82.5
DeepIM	18.38	0.1629	0.00825	0.1162	80.0

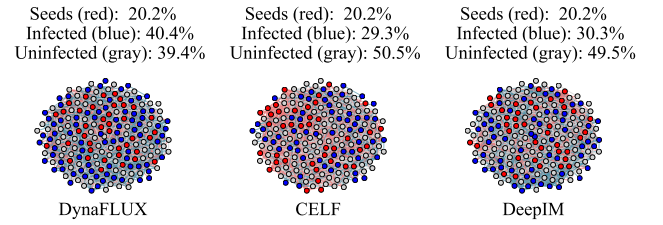
5.7 Case Study

Figure 6 visualizes the seed selection and final infection state on Jazz under the IC model, with a 22.2% seed ratio (i.e., 40 out of 198 nodes). Blue, red and gray nodes denote the initial seeds, infected nodes during influence spread, and uninfected nodes, respectively.

DynaFLUX's seeds produce a significantly larger infected component, covering 40.4% of the network, compared to 29.3% for CELf and 30.3% for DeepIM, demonstrating its practical effectiveness in identifying key spreaders using only timestamps. To understand this behavior, we compare the seed sets of DynaFLUX, CELf and DeepIM using the following structural metrics: core number c , degree centrality (DC), betweenness centrality (BC), neighbor Jaccard overlap J , and boundary ratio (BR). Here for each pair of nodes i, j , $J(i, j) = |N(i) \cap N(j)| / |N(i) \cup N(j)|$, and $BR(S) = |\{v \in S : \exists u \in N(v), C(u) \neq C(v)\}| / |S|$. Table 4 summarizes the aggregated values of these metrics for each method.

Though DynaFLUX does not minimize seed overlap, its selected nodes consistently exhibit higher core numbers and centrality measures, indicating a preference for structurally pivotal nodes. The

higher Jaccard overlap shows seeds share more neighbors, consistent with targeting densely connected hubs, while the high boundary ratio reveals placement at community interfaces to facilitate cross-community spread. In contrast, CELf and DeepIM implicitly favor minimal marginal redundancy, yielding more disperse but less central seeds. These confirm that DynaFLUX uses temporal-correlation attention to uncover key latent connections and prioritize nodes that maximize global diffusion, rather than pursuing overlap-minimization heuristics that yield suboptimal influence.

**Figure 6: Case study on Jazz (IC, 20.2% seeds).**

6 Conclusion

In this work, we presented DynaFLUX, a novel end-to-end framework for influence maximization that operates without access to network topology or predefined diffusion models. By leveraging a temporal-correlation self-attention encoder and policy-gradient optimization, DynaFLUX learns node embeddings and seed-selection policies directly from activation time-series. Extensive experiments on real-world and synthetic datasets show that DynaFLUX consistently outperforms classical and learning-based baselines under the Independent Cascade (IC) and Linear Threshold (LT) model, and remains competitive under the more challenging SIS setting.

Acknowledgments

This work is supported by the National Natural Science Foundation of China (Grants No U24B20143, No 72225012, No 72288101), and Civil Aviation Safety Supervision Capacity Building Project of Civil Aviation Administration of China (Grants No HA202511). This work is also supported by Beijing Engineering Research Center of Civil Aviation Big Data and sponsored by CAAI-MindSpore Open Fund, developed on OpenI Community.

References

- [1] Albert-László Barabási and Réka Albert. 1999. Emergence of scaling in random networks. *science* 286, 5439 (1999), 509–512.
- [2] Christian Borgs, Michael Brautbar, Jennifer Chayes, Brendan Lucier, et al. 2014. Maximizing social influence in nearly optimal time. In *Proceedings of the twenty-fifth annual ACM-SIAM symposium on Discrete algorithms*. SIAM, 946–957.
- [3] Tiantian Chen, Siwen Yan, Jianxiang Guo, Wei Wu, et al. 2023. ToupleGDD: A fine-designed solution of influence maximization by deep reinforcement learning. *IEEE Transactions on Computational Social Systems* 11, 2 (2023), 2210–2221.
- [4] Yanna Ding, Zijie Huang, Malik Magdon-Ismail, Jianxi Gao, et al. 2024. Predicting Time Series of Networked Dynamical Systems without Knowing Topology. *arXiv preprint arXiv:2412.18734* (2024).
- [5] P ERDdS and A R&wi. 1959. On random graphs I. *Publ. math. debrecen* 6, 290–297 (1959), 18.
- [6] Soheil Eshghi, Setareh Maghsudi, Valerio Restocchi, Sebastian Stein, Leandros Tassulas, et al. 2019. Efficient influence maximization under network uncertainty. In *IEEE INFOCOM 2019-IEEE Conference on Computer Communications Workshops (INFOCOM WKSHPS)*. IEEE, 365–371.
- [7] Manuel Gomez-Rodriguez, Jure Leskovec, Andreas Krause, et al. 2012. Inferring networks of diffusion and influence. *ACM Transactions on Knowledge Discovery from Data (TKDD)* 5, 4 (2012), 1–37.
- [8] Qintian Guo, Sibao Wang, Zhewei Wei, Ming Chen, et al. 2020. Influence maximization revisited: Efficient reverse reachable set generation with bound tightened. In *Proceedings of the 2020 ACM SIGMOD international conference on management of data*. 2167–2181.
- [9] Asela Hevathipathige, Qing Wang, Ahad N Zehmakan, et al. 2025. DeepSN: A Sheaf Neural Framework for Influence Maximization. In *Proceedings of the AAAI Conference on Artificial Intelligence*, Vol. 39. 17177–17185.
- [10] Katerina Hlaváčková-Schindler, Anna Melnykova, Irene Tubikanec, et al. 2024. Granger causal inference in multivariate hawkes processes by minimum message length. *Journal of Machine Learning Research* 25, 133 (2024), 1–26.
- [11] Wenjing Hong, Chao Qian, Ke Tang, et al. 2021. Efficient Minimum Cost Seed Selection With Theoretical Guarantees for Competitive Influence Maximization. *IEEE Transactions on Cybernetics* 51, 12 (2021), 6091–6104.
- [12] Harshavardhan Kamarthi, Priyesh Vijayan, Bryan Wilder, Balaraman Ravindran, Milind Tambe, et al. 2019. Influence maximization in unknown social networks: Learning policies for effective graph sampling. *arXiv preprint arXiv:1907.11625* (2019).
- [13] David Kempe, Jon Kleinberg, Éva Tardos, et al. 2003. Maximizing the spread of influence through a social network. In *Proceedings of the ninth ACM SIGKDD international conference on Knowledge discovery and data mining*. 137–146.
- [14] Sanjay Kumar, Abhishek Mallik, Anavi Khetarpal, Bhawani Sankar Panda, et al. 2022. Influence maximization in social networks using graph embedding and graph neural network. *Information Sciences* 607 (2022), 1617–1636.
- [15] John Boaz Lee, Ryan A Rossi, Sungchul Kim, Nesreen K Ahmed, Eunye Koh, et al. 2019. Attention models in graphs: A survey. *ACM Transactions on Knowledge Discovery from Data (TKDD)* 13, 6 (2019), 1–25.
- [16] Kristina Lerman, Aram Galstyan, et al. 2008. Analysis of social voting patterns on digg. In *Proceedings of the first workshop on Online social networks*. 7–12.
- [17] Jure Leskovec, Andreas Krause, Carlos Guestrin, Christos Faloutsos, Jeanne VanBriesen, Natalie Glance, et al. 2007. Cost-effective outbreak detection in networks. In *Proceedings of the 13th ACM SIGKDD international conference on Knowledge discovery and data mining*. 420–429.
- [18] Dexun Li, Meghna Lowalekar, Pradeep Varakantham, et al. 2021. CLAIM: Curriculum learning policy for influence maximization in unknown social networks. In *Uncertainty in Artificial Intelligence*. PMLR, 1455–1465.
- [19] Hui Li, Mengting Xu, Sourav S Bhowmick, Joty Shafiq Rayhan, Changsheng Sun, Jiangtao Cui, et al. 2022. PIANO: Influence maximization meets deep reinforcement learning. *IEEE Transactions on Computational Social Systems* 10, 3 (2022), 1288–1300.
- [20] Xiang Li, J David Smith, Thang N Dinh, My T Thai, et al. 2019. Tiptop:(almost) exact solutions for influence maximization in billion-scale networks. *IEEE/ACM Transactions on Networking* 27, 2 (2019), 649–661.
- [21] Chen Ling, Junji Jiang, Junxiang Wang, My T Thai, Renhao Xue, James Song, Meikang Qiu, Liang Zhao, et al. 2023. Deep graph representation learning and optimization for influence maximization. In *International conference on machine learning*. PMLR, 21350–21361.
- [22] Changan Liu, Changjun Fan, Zhongzhi Zhang, et al. 2024. Finding influencers in complex networks: An effective deep reinforcement learning approach. *Comput. J.* 67, 2 (2024), 463–473.
- [23] Andrew Kachites McCallum, Kamal Nigam, Jason Rennie, Kristie Seymore, et al. 2000. Automating the construction of internet portals with machine learning. *Information Retrieval* 3 (2000), 127–163.
- [24] Shodai Mihara, Sho Tsugawa, Hiroyuki Ohsaki, et al. 2015. Influence maximization problem for unknown social networks. In *Proceedings of the 2015 IEEE/ACM International Conference on Advances in Social Networks Analysis and Mining 2015*. 1539–1546.
- [25] Volodymyr Mnih, Adria Puigdomenech Badia, Mehdi Mirza, Alex Graves, Timothy Lillicrap, Tim Harley, David Silver, and Koray Kavukcuoglu. 2016. Asynchronous methods for deep reinforcement learning. In *International conference on machine learning*. PMLR, 1928–1937.
- [26] Hung T Nguyen, My T Thai, Thang N Dinh, et al. 2016. Stop-and-stare: Optimal sampling algorithms for viral marketing in billion-scale networks. In *Proceedings of the 2016 international conference on management of data*. 695–710.
- [27] George Panagopoulos, Fragkiskos D Malliaros, Michalis Vazirgiannis, et al. 2020. Multi-task learning for influence estimation and maximization. *IEEE Transactions on Knowledge and Data Engineering* 34, 9 (2020), 4398–4409.
- [28] George Panagopoulos, Nikolaos Tziortziotis, Michalis Vazirgiannis, Fragkiskos Malliaros, et al. 2023. Maximizing influence with graph neural networks. In *Proceedings of the International Conference on Advances in Social Networks Analysis and Mining*. 237–244.
- [29] Bastian Prasse, Piet Van Mieghem, et al. 2022. Predicting network dynamics without requiring the knowledge of the interaction graph. *Proceedings of the National Academy of Sciences of the United States of America* 119, 44 (Oct 2022).
- [30] Ryan Rossi, Nesreen Ahmed, et al. 2015. The network data repository with interactive graph analytics and visualization. In *Proceedings of the AAAI conference on artificial intelligence*, Vol. 29.
- [31] Richard S Sutton, David McAllester, Satinder Singh, and Yishay Mansour. 1999. Policy gradient methods for reinforcement learning with function approximation. *Advances in neural information processing systems* 12 (1999).
- [32] Jing Tang, Xueyan Tang, Xiaokui Xiao, Junsong Yuan, et al. 2018. Online processing algorithms for influence maximization. In *Proceedings of the 2018 international conference on management of data*. 991–1005.
- [33] Youze Tang, Yanchen Shi, Xiaokui Xiao, et al. 2015. Influence maximization in near-linear time: A martingale approach. In *Proceedings of the 2015 ACM SIGMOD international conference on management of data*. 1539–1554.
- [34] Youze Tang, Xiaokui Xiao, Yanchen Shi, et al. 2014. Influence maximization: Near-optimal time complexity meets practical efficiency. In *Proceedings of the 2014 ACM SIGMOD international conference on Management of data*. 75–86.
- [35] Cong Tran, Won-Yong Shin, Andreas Spitz, et al. 2024. IM-META: Influence maximization using node metadata in networks with unknown topology. *IEEE Transactions on Network Science and Engineering* 11, 3 (2024), 3148–3160.
- [36] Ashish Vaswani, Noam Shazeer, Niki Parmar, Jakob Uszkoreit, Llion Jones, Aidan N Gomez, Łukasz Kaiser, and Illia Polosukhin. 2017. Attention is all you need. *Advances in neural information processing systems* 30 (2017).
- [37] Duncan J Watts and Steven H Strogatz. 1998. Collective dynamics of ‘small-world’ networks. *nature* 393, 6684 (1998), 440–442.
- [38] Bryan Wilder, Nicole Immorlica, Eric Rice, Milind Tambe, et al. 2018. Maximizing influence in an unknown social network. In *Proceedings of the AAAI Conference on Artificial Intelligence*, Vol. 32.
- [39] Ronald J Williams. 1992. Simple statistical gradient-following algorithms for connectionist reinforcement learning. *Machine learning* 8, 3 (1992), 229–256.
- [40] Dongxia Wu, Tsuyoshi Idé, Georgios Kollias, Jiri Navrátil, Aurelie Lozano, Naoki Abe, Yian Ma, Rose Yu, et al. 2024. Learning Granger Causality from Instance-wise Self-attentive Hawkes Processes. In *International Conference on Artificial Intelligence and Statistics*. PMLR, 415–423.
- [41] Wenwen Xia, Yuchen Li, Jun Wu, Shenghong Li, et al. 2021. Deepis: Susceptibility estimation on social networks. In *Proceedings of the 14th ACM International Conference on Web Search and Data Mining*. 761–769.
- [42] Bo Yan, Kexiu Song, Jiamou Liu, Fanku Meng, Yiping Liu, Hongyi Su, et al. 2019. On the Maximization of Influence Over an Unknown Social Network.. In *AAMAS*, Vol. 19. 13–17.
- [43] Si Zhang, Hanghang Tong, Jiejun Xu, Ross Maciejewski, et al. 2019. Graph convolutional networks: a comprehensive review. *Computational Social Networks* 6, 1 (2019), 1–23.

A Appendix A: Algorithmic details

The training pipeline for DynaFLUX consists of two stages. First, when the true topology is not available, a surrogate graph is inferred from observed activation time-series using node-wise Lasso regression. Second, a policy for seed selection is trained via reinforcement learning on a single graph G , which may be either the true underlying network (for DynaFLUX+) or the surrogate inferred graph (for DynaFLUX). The reward signal is computed by simulating diffusion on G and used to update the policy with policy gradients augmented by entropy regularization.

Stage 1: Surrogate Topology Inference. Given only activation time-series, we construct a surrogate graph by fitting a separate Lasso

regression for each node to predict its next-step activation from the previous-step activations of all nodes. The resulting coefficient matrix $\hat{M}$ is converted into a graph G_{sim} , which serves purely as reward guidance during training. Although G_{sim} can differ substantially in structure from the true underlying network, the spread-based rewards computed on it remain highly correlated with those on the real graph; detailed empirical evidence and metrics appear in the main text in the “Validation of Reward Guidance” subsection. Because the activation time-series are noisy, we set the Lasso regularization weight to $\lambda = 50$ to aggressively suppress noise and retain the most salient predictive relationships (see Algorithm 2).

Algorithm 2 Surrogate Topology Inference via Lasso

Require: Time-series for one graph $\{x^{(0)}, \dots, x^{(T)}\} \subset \mathbb{R}^N$, regularization parameter λ (Lasso)

Ensure: Inferred surrogate adjacency matrix $\hat{M}$ and surrogate graph G_{sim}

1: Form lagged matrices:

$$X \leftarrow [x^{(0)}, \dots, x^{(T-1)}]^\top, \quad Y \leftarrow [x^{(1)}, \dots, x^{(T)}]^\top$$

2: Initialize $\hat{M} \leftarrow \mathbf{0}_{N \times N}$

3: **for** each node $i = 1, \dots, N$ **do**

4: Fit Lasso:

$$\hat{M}_{i,:} \leftarrow \arg \min_{m \in \mathbb{R}^N} \frac{1}{2} \|Y_{:,i} - Xm\|_2^2 + \lambda \|m\|_1$$

5: **end for**

6: $G_{\text{sim}} \leftarrow \text{GraphFromMatrix}(\hat{M})$

7: **return** $\hat{M}, G_{\text{sim}}$

Stage 2: Policy Learning with Diffusion-based Reward. Using a single graph G (either the true topology or the inferred surrogate), the policy π_θ selects seeds sequentially. The function $f(\cdot)$ in Equation (8) denotes concatenation. After each seed addition, we simulate diffusion on G to estimate marginal influence gains, which are converted into advantages using a running baseline. We update the policy via a REINFORCE-style gradient augmented with entropy regularization to sustain exploration. Training proceeds for 300 epochs. At each selection step, marginal gains are estimated by Monte Carlo diffusion—using $R = 100$ runs during training to reduce variance and $R = 10$ runs during evaluation to align with DeepIM’s protocol. The diffusion model (IC, LT, or SIS) varies by experiment. The running baseline updates with decay factor $\alpha = 0.9$. The entropy weight is scheduled as

$$\beta_{\text{ent}} = \max(0.05(1 - e/E), 0.001), \quad (15)$$

balancing early exploration and later convergence. We scale advantages (e.g., multiply by 100) before computing the policy gradient loss (see Algorithm 1).

Why Direct Joint-Probability Backpropagation Fails. An alternative is to model the joint influence probability of a seed set S by first estimating pairwise influence probabilities $p_{u \rightarrow v}$ via a neural network and then computing

$$P(v | S) = 1 - \prod_{u \in S} (1 - p_{u \rightarrow v}). \quad (16)$$

Table 5: The Overview of Dataset.

Dataset	Cora-ML	NS	PG	Jazz	Digg	SD
Nodes	2,810	1,461	4,941	198	279,630	50,000
Edges	7,981	2,742	6,594	2,742	2,251,166	250,000

While this objective is differentiable, the product $\prod_{u \in S} (1 - p_{u \rightarrow v})$ leads to exponentially small gradients as $|S|$ or n grows, causing severe vanishing-gradient issues even under a log-transform. To overcome this, we instead employ Monte Carlo-based policy gradients, which propagate learning signals proportional to observed spread rather than infinitesimal derivatives of a product of probabilities.

B Appendix B: Experimental Details

B.1 Experimental Setup

Datasets. We use five real-world datasets and one synthetic dataset to evaluate DynaFLUX: (1) Jazz [30], a collaboration network, where nodes represent jazz musicians and edges indicate co-performance in bands; (2) Cora-ML [23], a directed citation network of machine learning papers, where directed edges denotes citations links; (3) Power Grid (PG) [30], the topology of the U.S. Western States power grid, where nodes denote generators, transformers, or substations, and edges represent transmission lines; (4) Network Science (NS)[30], a co-authorship network among scientists working on network theory; (5) Digg [16], a directed social network, where follower edges enable content propagation; and (6) a synthetic dataset (SD) [21] consisting of Erdős-Rényi (ER) networks. Table 5 summarizes their statistics.

Implementation details. All experiments were conducted on a workstation with an Intel Xeon Gold 6148 CPU (80 cores, 2.4GHz), an NVIDIA Tesla V100 SXM2 GPU (32 GB), and 256 GB of RAM. For reproducibility, we fixed the random seed to 42. (1) For the IC model, we adopt the weighted-cascade setting, where the propagation probability on each edge (u, v) is set to $p_{u,v} = 1/d_v^{\text{in}}$, where d_v^{in} is the in-degree of node v . (2) For the LT model, the threshold θ_v for each node v is uniformly sampled from the interval $[0.3, 0.6]$. (3) For the SIS model, we employ the susceptible–infected–susceptible process with transmission probability $\beta = 0.001$ and recovery probability $\lambda = 0.001$; at each time step, an infected node transmits the infection to each susceptible neighbor independently with probability β , and recovers (becoming susceptible again) with probability λ .

A fundamental step for DynaFLUX, during both training on synthetic graphs and evaluation on real networks, is generating node-level time series data. In real scenarios (e.g., a single observed epidemic outbreak), only one diffusion trace is available, preventing repeated sampling. Moreover, a pure diffusion operator would rapidly equalize all node states, and remove informative structural signals.

To address these issues, we simulate a heat-diffusion process augmented with random pulses as follows. At each time step, each node’s current “heat” spreads uniformly to its neighbors, and short, high-amplitude pulses are injected into a small random subset of nodes, together with Gaussian noise.

Formally, given a directed graph G , let A be the adjacency matrix of G , and let $\text{diag}(1/\text{deg})$ be the degree-normalization matrix. Given a retention coefficient r and a diffusion coefficient d , we define the update matrix:

$$M = rI + dA \text{diag}(1/\text{deg}). \quad (17)$$

In our experimental setup, the retention coefficient r and diffusion coefficient d are both chosen to be 0.5. Denote by $\mathbf{x}(t) \in \mathbb{R}^N$ the node-state vector at step t . We initialize $\mathbf{x}(0)$ with a large random pulse α on all nodes. Then at each step $t \in [0, T-1]$, we update:

$$\mathbf{x}(t+1) = M(\mathbf{x}(t) + \boldsymbol{\eta}(t) + \sum_{i \in S_t} \epsilon_i(t)), \quad (18)$$

where S_t is a random subset of nodes for pulsing, $\epsilon_i(t) \sim \mathcal{N}(0, \sigma^2 I)$ is the node-specific pulse noise, and $\boldsymbol{\eta}(t) \sim \mathcal{N}(0, \sigma^2 I)$ denotes global noise. The resulting matrix $X \in \mathbb{R}^{N \times T}$, whose columns are $\mathbf{x}(t)$, serves as the input time-series used for both training on synthetic graphs and evaluation on real networks.

To further examine whether the policy guidance depends critically on the specific reconstruction scheme, we also implemented three correlation-based alternatives, each transforming pairwise correlations of node activations into a surrogate adjacency.

Epsilon-thresholded Graph (Epsilon). Edges are included whenever the absolute Pearson correlation between two node time-series exceeds a threshold ε . Formally, $A_{ij} = 1$ if $|\text{corr}(x_i, x_j)| \geq \varepsilon$, and $A_{ij} = 0$ otherwise. This approach yields an unweighted and typically sparse undirected graph. In experiments, we set the threshold to $\varepsilon = 0.25$, which approximately matches the density of the Lasso-inferred topology and balances noise suppression with connectivity.

K-Nearest Neighbors Graph (KNN). Here, each node connects to its k most correlated peers, ensuring a fixed out-degree per node and a uniformly sparse structure. The adjacency matrix is constructed by setting $A_{ij} = 1$ if j belongs to the top- k entries of $|\text{corr}(x_i, x_j)|$, and symmetrizing A by averaging with its transpose. In our experiments, we used $k=10$ (roughly 10% of N).

Probabilistic Graph (Prob). This variant defines a soft, differentiable random graph whose edge probabilities are proportional to correlation magnitudes. Specifically, each pair (i, j) is assigned an inclusion probability $p_{ij} = |\text{corr}(x_i, x_j)|^\gamma$, and edges are sampled according to independent Bernoulli draws. We used $\gamma=1.2$ to mildly sharpen high correlations while keeping stochastic variability, and fixed the random seed for reproducibility. This probabilistic construction mimics a “soft topology” and can be viewed as the expected adjacency under the empirical correlation distribution.

C Appendix C: Additional Analyses and Experiments

C.1 Realism of the Diffusion–Pulsing Process and Real-Data Validation

The diffusion–pulsing generator (Eq. (19)) is designed to capture two common properties of real temporal diffusion: (i) a slow background diffusion process and (ii) irregular bursts caused by exogenous shocks (e.g., trending events). Importantly, real temporal observations are overwhelmingly single-trajectory: in realistic settings we typically observe one unfolding multivariate time series

Table 6: Influence spread (%) on SNAP–CollegeMsg with 20 time intervals.

Seed budget (k)	0.01	0.05	0.10	0.20
IC_inf (%)	55.0	58.0	62.8	71.0
LT_inf (%)	15.0	99.7	99.7	99.7
SIS_inf (%)	30.4	37.2	39.8	43.4

rather than repeated independent cascades. Although Algorithm 2 operates on a single long trajectory, at each timestep a random subset of nodes receives exogenous pulses. Therefore, the trajectory implicitly contains many independent excitation–propagation episodes, so Lasso effectively observes an ensemble of cascades embedded in one time series, mitigating instability typical of true single-trajectory learning.

To further validate performance beyond synthetic generation, we evaluate on the SNAP–CollegeMsg temporal network. We partition the timeline into 20 intervals and use each node’s activity count per interval as the node-level time series. We report the influence spread (in %) under IC/LT/SIS diffusion for seed budgets $k \in \{0.01, 0.05, 0.10, 0.20\}$. Table 6 summarizes the results on SNAP–CollegeMsg.

C.2 Sensitivity to the Number of Monte-Carlo Simulations R

We evaluate the reward variance under different Monte-Carlo runs $R \in \{5, 10, 20, 50, 100, 150, 200\}$ on a large real-world graph setting (Digg, IC, seed budget $k = 0.01$). Across all tested R , the estimated mean reward is very stable (fluctuation $< 0.2\%$), while the standard deviation remains tiny relative to the total cascade scale (about 1.68×10^5), indicating that moderate R already yields sufficiently low-variance reward estimates and further increasing R offers limited practical benefit.

This observation is consistent with the inherent stability of the IC diffusion on large real-world graphs: (i) even with small R , the Monte-Carlo noise is practically negligible; (ii) this is expected since the cascade activates a massive number of nodes, and the resulting “averaging effect” (law of large numbers) overwhelms sampling noise, making the spread estimate highly concentrated.

Table 7: Graph structural characteristics for failure-mode diagnosis. DegCV = coefficient of variation of degree.

Graph	AvgDeg	DegCV	DegGini	Clustering	Modularity
Cora-ML	5.6804	1.4879	0.4965	0.2777	0.7600
Network Science	3.7536	0.9212	0.4153	0.6937	0.9592
Power Grid	2.6691	0.6711	0.3248	0.0801	0.9358
Jazz	27.6970	0.6286	0.3460	0.6175	0.4430
Synthetic	10.0004	0.3164	0.1773	0.000187	0.2537
Digg	11.0727	6.2969	0.8422	0.0924	0.4604

C.3 Structural Failure-Mode Analysis and Runtime Breakdown

Graph-structure analysis and failure mode. We quantify structural characteristics of all graphs, including average degree, degree

Table 8: Module-level time (seconds) across datasets and budgets. Enc = encoder, Dec = decoder, Sim = simulation.

Graph	$k = 0.01$			$k = 0.05$			$k = 0.10$			$k = 0.20$		
	Enc	Dec	Sim	Enc	Dec	Sim	Enc	Dec	Sim	Enc	Dec	Sim
Network Science	0.0769	0.1337	0.7782	0.0788	0.1406	0.8074	0.0784	0.1622	0.7966	0.0785	0.2031	0.8270
Jazz	0.0013	0.0008	0.1660	0.0011	0.0031	0.1571	0.0012	0.0051	0.1770	0.0016	0.0114	0.1615
Cora-ML	0.0016	0.0080	1.6977	0.0020	0.0355	1.7262	0.0021	0.0706	1.7593	0.0017	0.1435	1.8522
Synthetic	0.0040	0.2333	37.8230	0.0038	0.8658	41.9374	0.0039	1.5308	41.4387	0.0041	2.9994	44.8990
Digg	0.0054	2.8894	338.0341	0.0053	12.7110	332.6015	0.0080	24.9528	344.2733	0.0052	49.4519	339.7504
Power Grid	0.0011	0.0133	2.5841	0.0013	0.0658	2.7812	0.0010	0.1226	2.7101	0.0010	0.2657	2.7741

heterogeneity, clustering coefficient, and modularity. The quantified graph statistics are reported in Table 7. The analysis shows that the only dataset where DynaFLUX is noticeably weaker is Synthetic (Erdős–Rényi random graph), which lacks exploitable structure: (i) the degree distribution is extremely uniform (DegCV = 0.316, Deg-Gini = 0.177), implying almost no hubs; (ii) the average clustering is near zero (Clustering = 1.87×10^{-4}), indicating little local structure; (iii) the modularity is low (0.25), suggesting very weak community organization; (iv) the graph is close to purely random connections. In such “structure-missing” networks, diffusion outcomes across nodes are nearly exchangeable and there is no consistent structural pattern for any method to exploit. Consequently, RL-based methods, GNN-based methods, and classical greedy IM baselines all yield similar performance, and DynaFLUX’s structure-guided advantage becomes less visible.

Runtime. We report module-level wall-clock time (seconds) for encoder/decoder/simulation under different seed budgets. The module-level runtime breakdown is provided in Table 8. A consistent pattern

is observed: the encoder is fast, the decoder cost grows with the budget, and Monte-Carlo simulation dominates end-to-end runtime. This breakdown explains why DynaFLUX remains efficient in representation learning, while the overall runtime is primarily determined by diffusion evaluation procedures.

linear attention. We replace standard self-attention with a kernelized linear attention to avoid constructing the $N \times N$ attention matrix and reduce the encoder-side complexity from $O(N^2)$ to roughly linear in N . In our implementation, node embeddings are projected to Q, K, V , and we apply a positive kernel feature map $\phi(\cdot) = \text{ELU}(\cdot) + 1$ so that attention can be computed via global summaries (e.g., $\sum \phi(K)V$ and $\sum \phi(K)$) instead of pairwise interactions. This makes the encoder more scalable, but the end-to-end runtime improves only modestly because Monte-Carlo diffusion simulation remains the dominant bottleneck. Empirically, linear attention leads to only a small performance drop (about 1–2%), which we find to be a reasonable trade-off.